UNIT–IV

Markov Decision Process: MDP formulation, utility theory, utility functions, value iteration, policy iteration, and partially observable MDPs.

//By Manish D@nge

Markov Decision Process

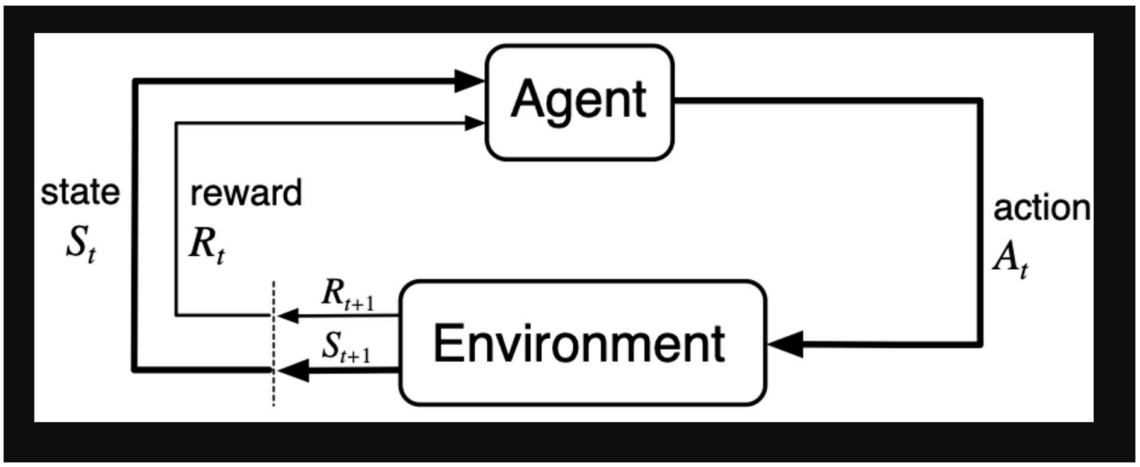
A Markov Decision Process (MDP) models decision-making where outcomes are partly random and partly under the agent's control. In reinforcement learning, MDPs are solved to find an optimal policy that maximizes expected rewards over time using algorithms like Value Iteration or Policy Iteration.

A thorough introduction to Markov Decision Processes (MDPs), a mathematical framework for stochastic decision-making in dynamic systems, is given in this article. Under the guidance of the Markov Property, which depends only on the current state and does not require historical data, MDPs allow decision-makers or intelligent agents to select the best course of action based on current states and rewards.

Markov Decision Process (MDP) is a way to describe how a decision-making agent like a robot or game character moves through different situations while trying to achieve a goal. MDPs rely on variables such as the environment, agent's actions and rewards to decide the system's next optimal action. It helps us answer questions like:

- What actions should the agent take?
- What happens after an action?
- Is the result good or bad?

In artificial intelligence Markov Decision Processes (MDPs) are used to model situations where decisions are made one after another and the results of actions are uncertain. They help in designing smart machines or agents that need to work in environments where each action might led to different outcomes.



Key Components of an MDP

An MDP has five main parts:

Components of Markov Decision Process

1. States (S): A state is a situation or condition the agent can be in. For example, A position on a grid like being at cell (1,1).

2. Actions (A): An action is something the agent can do. For example, Move UP, DOWN, LEFT or RIGHT. Each state can have one or more possible actions.

3. Transition Model (T): The model tells us what happens when an action is taken in a state. It's like asking: "If I move RIGHT from here, where will I land?" Sometimes the outcome isn't always the same that's uncertainty. For example:

- 80% chance of moving in the intended direction
- 10% chance of slipping to the left
- 10% chance of slipping to the right

This randomness is called a stochastic transition.

4. Reward (R): A reward is a number given to the agent after it takes an action. If the reward is positive, it means the result of the action was good. If the reward is negative it means the outcome was bad or there was a penalty help the agent learn what's good or bad. Examples:

- +1 for reaching the goal
- -1 for stepping into fire
- -0.1 for each step to encourage fewer moves

5. Policy (π): A policy is the agent's plan. It tells the agent: "If you are in this state, take this action." The goal is to find the best policy that helps the agent earn the highest total reward over time.

Applications

Markov Decision Processes are useful in many real-life situations where decisions must be made step-by-step under uncertainty. Here are some applications:

- **Robots and Machines:** Robots use MDPs to decide how to move safely and efficiently in places like factories or warehouses and avoid obstacles.
- **Game Strategy:** In board games or video games MDPs help characters to choose the best moves to win or complete tasks even when outcomes are not certain.
- **Healthcare:** Doctors can use it to plan treatments for patients, choosing actions that improve health while considering uncertain effects.
- **Traffic and Navigation:** Self-driving cars or delivery vehicles use it to find safe routes and avoid accidents on unpredictable roads.
- **Inventory Management:** Stores and warehouses use MDPs to decide when to order more stock so they don't run out or keep too much even when demand changes.

Markov Decision Process Examples

In a number of application domains, including computer science, electrical engineering, manufacturing, operations research, finance and economics, telecommunications, and so on, MDPs have made substantial contributions.

Here are a few straightforward instances when MDP is still very much in play:

1. Routing Problems

Routing issues like those identified in the travelling salesman problem (TSP) are handled using MDP-based sequential decision-making. There are several components that make up TSP:

- Salesman = agent,
- The agent's available routes are the things it can do in its current condition.
- Rewards are equal to the expenses of using the particular routes, and

- Goal: the best course of action that reduces the total cost over the course of the journey.

2. Overseeing the Upkeep and Restoration of Dynamic Systems

Dynamic systems that deteriorate over time as a result of actions in various contexts, such as cars, buses, or trucks, are subject to maintenance and repair issues. The choices in this instance could be to replace the vehicle's essential parts, carry out repairs, or do nothing at all. The system can select measures that help reduce the vehicle's maintenance costs over time thanks to the formulation of this problem under MDP.

3. Designing Intelligent Machines

MDPs are widely used in robotics, complex autonomous systems, autonomous vehicles, automated systems, and other fields as a result of developments in AI and machine learning. MDPs are employed in reinforcement learning models to teach machines and robots how to learn on their own and complete particular tasks.

For instance, DeepMind Technologies, a division of Google, trains computer systems to outperform humans in Atari games by fusing neural networks with the MDP architecture. Additionally, the business has trained machines to play board games like AlphaGo using MDPs. The MDP framework has also been used by DeepMind Technologies to train simulated robots to run and walk.

4. Designing Quiz Games

Designing quiz games with certain levels has been a common application of MDPs. There is a question at every level that, if answered correctly, yields a financial reward. The questions get harder and offer greater rewards as the level rises.

When a player performs well in a quiz game and properly answers every question, they receive a prize and have the option to continue playing the quiz or end the game. The participant may be able to keep all of the prizes he has accrued if he decides to stop. He forfeits all of the accrued incentives, though, if he decides to participate and is unable to correctly answer the question at a particular level. These games aim to maximise the benefits while evaluating the actions of playing or stopping.

5. Managing Wait Time at a Traffic Intersection

The duration of traffic signals is determined with the aid of MDPs. Here, the goal is to reduce wait times for cars while increasing the number of vehicles that can pass through intersections. It may be a two-way intersection, meaning that traffic can move either way, for example, south or west. Additionally, it is anticipated that the system has sensors that provide information on how many cars are approaching the intersection. In this instance, the traffic lights are green and red. Every step in this case takes a few

seconds, either two or five. Depending on the number of cars at the intersection and their wait durations, you can use MDPs to determine whether or not to alter the traffic lights.

6. Determining the Number of Patients to Admit to a Hospital

A specific number of patients come to the hospital each day for a variety of reasons. After that, the hospital must determine how many patients it can admit while taking into account variables like:

- the quantity of patients admitted thus far,
- how many beds are available, and
- the total number of patients per day that recover and are released.

What is Utility Theory?

Utility theory in artificial intelligence is a mathematical framework used to model decision-making under uncertainty. It allows one to assign subjective values or preferences to different outcomes and helps make optimal choices based on these values. Utility theory is widely used in various AI applications such as **game theory**, **economics**, **robotics**, and **recommendation systems**, among others.

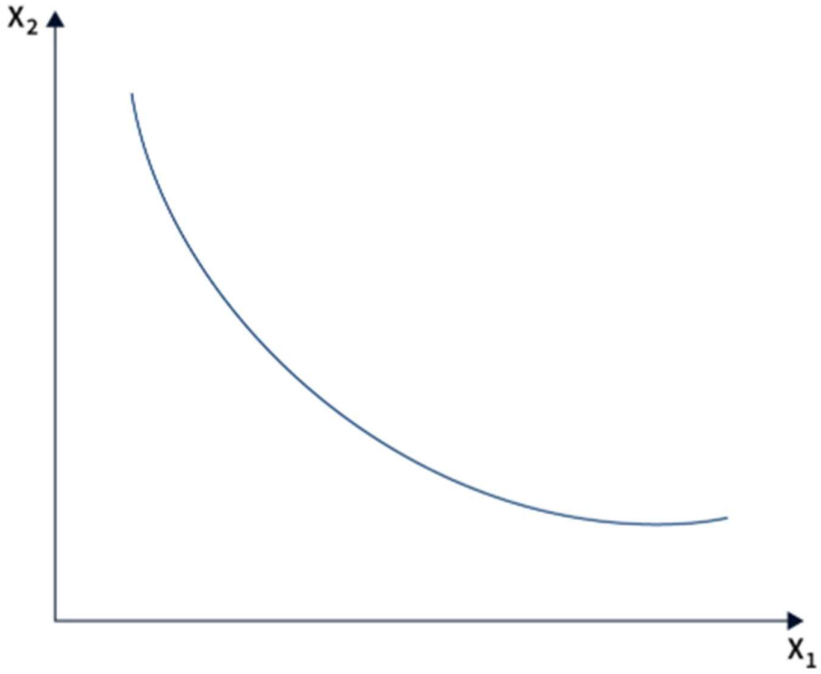
At its core, utility theory helps AI systems make decisions that maximize a specific goal, referred to as **utility**. The concept of utility is **subjective** and varies from person to person or from system to system. It represents the degree of satisfaction associated with different outcomes. For example, in a **recommendation system**, the utility could describe the level of user satisfaction with a particular recommendation. In a **robotics application**, a utility could represent the cost or risk of different actions.

Utility theory also provides a way to **model decisions in uncertain or probabilistic environments**, where the outcomes are associated with different probabilities. For example, in a **game of poker**, the utility of a particular action may depend on the probabilities of different cards being dealt to the player. We can use the utility function to calculate the expected utility of each action, which is the average utility weighted by the corresponding probabilities. The AI system can then choose the action with the **highest expected utility**.

What is Utility Function?

A **utility function** is a mathematical function used in Artificial Intelligence (AI) to represent a system's preferences or objectives. It assigns a numerical value, referred to as **utility**, to different outcomes based on their **satisfaction level**. The utility function is a **quantitative measure** of the system's subjective preferences. It is used to guide decision-making in AI systems. An agent or system typically defines the utility function based on its **goals**, **objectives**, and **preferences**. It maps different outcomes to their corresponding utility values, where **higher utility values represent more desirable outcomes**. The utility function is subjective and can vary from one agent or system to another, depending on the specific context or domain of the AI application.

The utility function plays a crucial role in decision-making in AI systems. It allows the AI system to compare and rank different outcomes or actions based on their utility values and choose the one with the highest utility. The choice of action with the highest utility depends on the system's objectives, as reflected in the utility function.



Utility Function Representation (denoted by U)

The utility function is typically denoted as U. It is a mathematical function that takes as input the different features of an outcome and maps them to a real-valued utility value. We can represent the utility function mathematically as $U(x)$, where x represents the attributes or features of an outcome. How we define the utility function can vary depending on the application and the type of decision problem we are trying to solve.

Decision Making

One common approach in AI decision-making is to **maximize the expected utility**, which considers the probability of different outcomes occurring. The expected utility is calculated by multiplying the utility of each outcome by its corresponding probability and summing up the results. The AI system chooses the **action with the highest expected utility** as the **optimal choice**.

Examples:

1. **Self-Driving Cars:** In the **self-driving cars** application, the utility function may consider factors such as **time taken**, **fuel consumption**, **safety**, and **comfort**, and assign utility values to different routes based on these factors. The self-driving car can then use the utility values to calculate the expected utility of each route, taking into account the probabilities of different traffic conditions or road obstacles, and choose the route with the highest expected utility to reach the destination.
2. **Recommendation Systems:** Consider a **recommendation system** that suggests movies to users based on their preferences. The utility function of the recommendation system may assign higher utility values to movies that match the **user's preferred genre, actors, or directors** and lower utility values to films that do not match these preferences. The recommendation system can then use the utility values to rank and recommend movies to the user based on their utility values, with higher utility movies being recommended more prominently.

Utility theory

Utility theory in artificial intelligence provides a formal framework for reasoning about decision-making under uncertainty. It is often used in AI systems to model decision-

making in situations where outcomes are uncertain or probabilistic, and the AI system needs to make choices based on its preferences or subjective values.

Lottery

To understand the concept of utility theory in artificial intelligence, let's consider a simple example of a lottery. Suppose you are given the option to play a lottery with two choices:

1. A guaranteed prize of $$100$
2. A 50% chance of winning $$200$ and a 50% chance of winning nothing

Which option would you choose? Your decision depends on your **risk tolerance**, **financial situation**, and **personal preferences**. Utility theory provides a way to model and quantify these preferences mathematically using a utility function.

Notation

Let us define some basic notation commonly used in utility theory:

- Let x represent an outcome or option.
- Let $U(x)$ denote the utility function, which maps x to its utility value.
- Let $p(x)$ denote the probability of outcome x occurring.
- Let $E[U(x)]$ denote the expected utility of outcome x, which is the sum of the utility values of all possible outcomes weighted by their respective probabilities.

The $E[U(x)]$ is calculated using the following expression:

$$E[U(x)] = \sum_i \quad P(x_i) \cdot U(x_i)$$

In this formula, the $E[U(x)]$ represents the expected utility of a decision or action, which is the sum of the product of the probability of each outcome x_i (denoted as $P(x_i)$) and its corresponding utility value (denoted as $U(x_i)$). The summation is taken over all possible outcomes i.

Diving Into Utility Theory and MEU

One of the fundamental concepts in utility theory in artificial intelligence is the idea of **Maximum Expected Utility (MEU)**. MEU is a decision-making principle that suggests choosing the option that maximizes the expected utility. In other words, an AI system should select the option that is expected to yield the highest utility value, taking into account the probabilities of different outcomes.

Utility Theory Axioms

Utility theory in artificial intelligence is based on a set of axioms or principles that define the properties of a rational utility function. These axioms serve as the foundation for understanding how we can use utility functions to model decision-making under uncertainty. Let's explore some of the key utility theory axioms:

Conclusion

- Utility theory in artificial intelligence plays a crucial role in decision-making by providing a mathematical framework for incorporating subjective values into the **decision-making process**.
- Using utility functions and the principles of maximum expected utility, AI systems can choose different options based on their perceived value or utility, considering factors such as **risk**, **uncertainty**, and **subjective preferences**.
- The axioms of utility theory define the properties of a rational utility function, ensuring that the preferences modeled by the utility function are **consistent**, **well-behaved**, and enable **rational decision-making**.
- Overall, utility theory provides a powerful tool for AI systems to make informed and rational decisions in complex and uncertain environments, making it an essential concept in the field of artificial intelligence.

Utility Theory vs Utility Function (Difference Table)

Point	**Utility Theory**	**Utility Function**
Meaning	A general theory about how agents make the best decisions	A mathematical formula used to measure how good each outcome is
Role	Provides the *idea and principles* of rational decision-making	Converts outcomes into *numbers (utilities)*
Purpose	Helps AI choose the best action based on preferences and outcomes	Gives a score so the AI can compare different actions
Level	High-level concept	Low-level mathematical tool
Focus	Why and how an agent should choose	How to assign a value to outcomes
Used for	Understanding decision strategy	Calculating and ranking choices

Point	Utility Theory	Utility Function
Output	Decision rules or guidelines	Utility values (numbers)
Depends on	Agent's goals, preferences, rationality	Input variables like rewards, risk, cost, probability
Example	"Choose the option that maximizes happiness."	U(x) = 3x + 5 or U(outcome) = probability × reward
Where used in AI	Planning, decision theory, rational agents	Game AI, RL, self-driving optimization
Analogy	The rulebook on choosing best options	The scorecard used for ranking those options

Value Iteration vs. Policy Iteration

Value Iteration and Policy Iteration are two popular techniques used in dynamic programming to solve Markov Decision Processes (MDPs). Both methods aim to find the best possible strategy known as the *op*timal policy for an agent to follow in a given environment. Understanding the differences, strengths and weaknesses of these two methods is important to choose the right approach for specific RL problems.

✅ **Value Iteration and Policy Iteration (Deep + Simple Explanation)**

Value Iteration and **Policy Iteration** are two core algorithms used in Reinforcement Learning to solve **Markov Decision Processes (MDPs)**. Their goal is the same:
👉 *Find the best policy (best action in every state) that gives maximum long-term reward.*

1. Value Iteration

Value Iteration focuses on learning the **value of each state** directly.
It repeatedly updates the value of a state using the Bellman Optimality Equation.
In every step, the algorithm picks:

- the **best possible action**,
- calculates the expected reward,
- and updates the value of the state.

This continues until values stop changing (converge).
Once values are stable, the policy is extracted by choosing the action with the highest value.
Value Iteration is fast and works well when the environment is simple.

What is Value Iteration?

- Value Iteration is an iterative algorithm used to compute the optimal value function $V^*(s)$ for each state s in an MDP. The value function is a measure of the expected return (reward) from a given state under the optimal policy.
- In Value Iteration the Bellman Optimality Equation is used to iteratively update the value of each state until it converges to the optimal value function:

$$V^*(s) = \max_a [R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s')]$$

Where:

- $R(s,a)$ is the immediate reward,
- $P(s' \mid s,a)$ is the transition probability,
- γ is the discount factor and
- s' represents the next state.

value iteration network

Once the value function converges, the optimal policy can be derived by selecting the action a that maximizes the value function:

$$\pi^*(s) = \arg\max_a [R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s')$$

What is Policy Iteration?

Policy Iteration is another dynamic programming algorithm used to compute the optimal policy. It alternates between two steps:

Policy Iteration

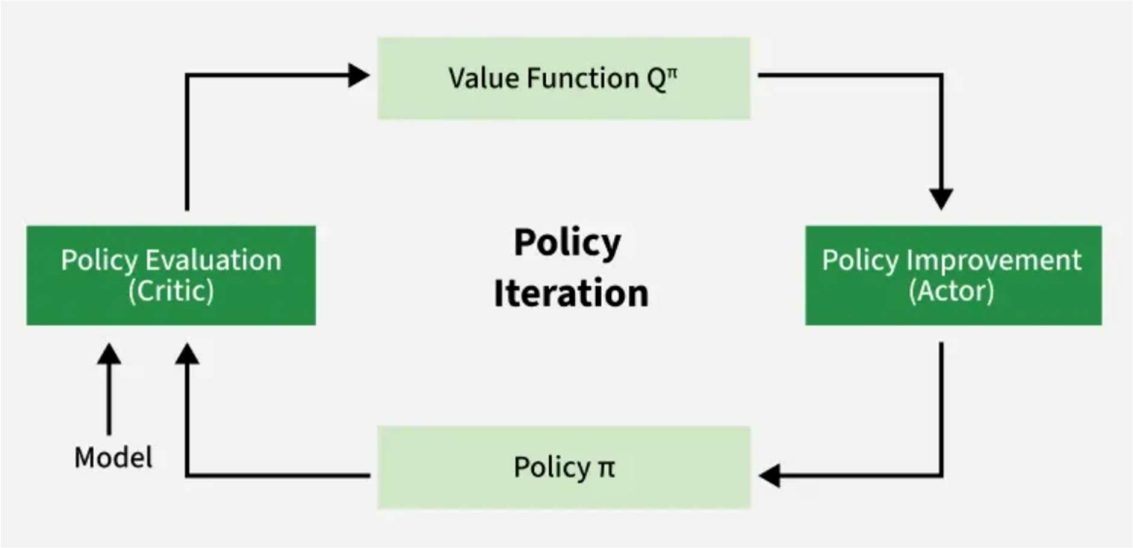
- **Policy Evaluation**: For a given policy π, the value function $V^{\pi}(s)$ is computed using the Bellman Expectation Equation:

$$V^{\pi}(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) V^{\pi}(s')$$

- **Policy Improvement**: Once the value function for the current policy is calculated the policy is updated to improve it by selecting the action that maximizes the expected return from each state:

$$\pi'(s) = \arg\max_{a} [R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^{\pi}(s')]$$

This process repeats until the policy converges meaning it no longer changes between iterations.



2. Policy Iteration

Policy Iteration works in **two steps** repeatedly:

1. **Policy Evaluation** → Calculate how good the current policy is by computing state values.
2. **Policy Improvement** → Improve the policy by choosing better actions based on those values.

This loop continues until the policy stops changing.
Policy Iteration is often slower per step but usually converges in fewer iterations.

Comparison Between Value Iteration and Policy Iteration

Feature	Value Iteration	Policy Iteration
Approach	Updates the value function iteratively until convergence.	Alternates between policy evaluation and policy improvement.
Convergence	Converges when the value function converges.	Converges when the policy stops changing.
Computational Cost	More computationally expensive per iteration due to full evaluation of all states.	Requires more iterations but may converge faster in terms of fewer iterations.
Policy Output	The policy is derived after the value function has converged.	The policy is updated during each iteration.

Feature	Value Iteration	Policy Iteration
Speed of Convergence	May require many iterations for convergence, especially in large state spaces.	Tends to converge faster in practice, especially when the policy improves significantly at each iteration.
State Space	Typically suited for smaller state spaces due to computational complexity.	Can handle larger state spaces more efficiently.

Other definition POMDP:

Partially Observable Markov Decision Processes (POMDPs) are a mathematical fra mework used in AI for decision-making under uncertainty, where the agent has incomplete information about the e nvironment.

Definition and Components

A POMDP is an extension of a Markov Decision Process (MDP) that accounts for situ ations where the agent cannot fully observe the current state of the environment. It is formally defined by the tuple $(S, A, T, R, \Omega, O, \gamma)$, where:

- **S: Set of states in the environment.**
- **A: Set of possible actions the agent can take.**
- **T: Transition probability function $T(s' \mid s, a)$ that defines the probability of moving to state s' from state s after taking action a.**
- **R: Reward function $R(s, a)$ that assigns a numerical reward for taking action a in state s.**

- **Ω: Set of observations the agent can perceive.**
- **O: Observation function $O(o \mid s', a)$ that defines the probability of observing o after taking action a and ending up in state s'.**

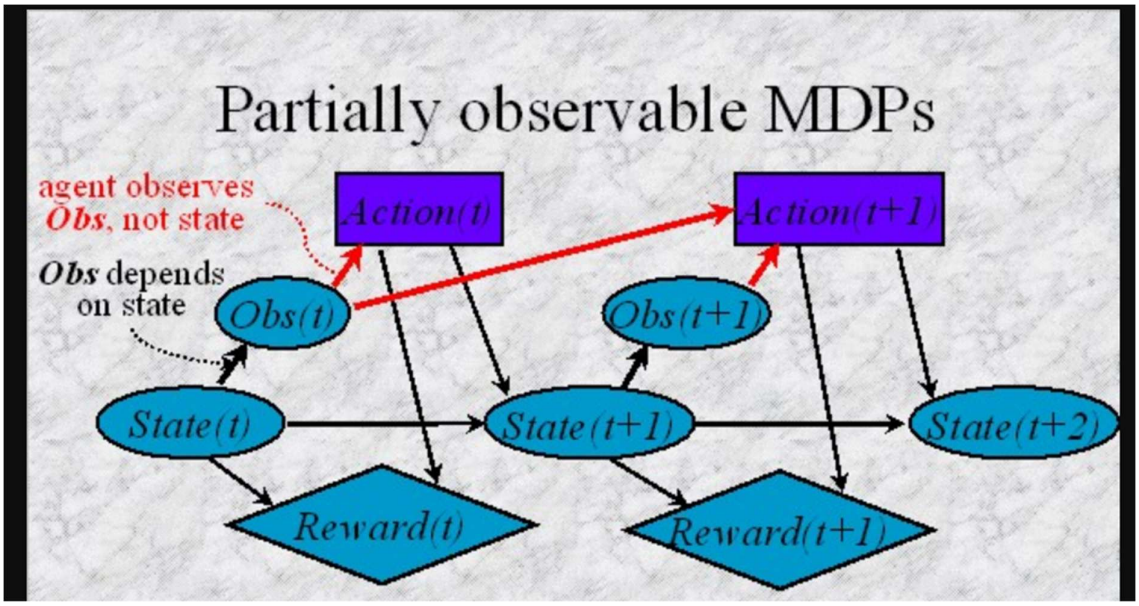
Partially Observable Markov Decision Process (POMDP) in AI

Partially Observable Markov Decision Process (POMDP) is a mathematical framework employed for decision-making in situations of uncertainty, where the decision-maker lacks complete information or noisy information regarding the current state of the environment. POMDPs have broad applicability in diverse domains such as robotics, healthcare, finance, and others.

This article provides an in-depth overview of Partially Observable Markov Decision Processes (POMDPs), their components, mathematical framework, solving strategies, and practical application in maze navigation using Python.

Pre-Requisites

1. **Probability theory:** Probability theory is applied to POMDPs to model the uncertainty surrounding the observations made by the agent and the changes in state within the environment.
2. **Markov processes:** A Markov process, sometimes referred to as a Markov chain, is a stochastic model that depicts how a system changes over time. It assumes that the system's future state is solely dependent on its current state and not on the preceding set of events.
3. **Decision theory**: Taking into account the trade-offs between various actions and their possible outcomes, decision theory offers a framework for making decisions under uncertainty.



What is Partially Observable Markov Decision Process (POMDP)?

A POMDP models decision-making tasks where an agent must make decisions based on incomplete or uncertain state information. It is particularly useful in scenarios where the agent cannot directly observe the underlying state of the system but rather receives observations that provide partial information about the state.

Components of a POMDP

A POMDP is formally defined by the following elements:

- **States (S)**: A finite set of states representing all possible conditions the system can be in.
- **Actions (A)**: A finite set of actions available to the agent.
- **Transition Model (T)**: A function $T(s, a, s') = P(s' \mid s, a)$ that defines the probability of transitioning from state *s* to state *s'* under action ?*a*.
- **Observations (O)**: A finite set of observations that the agent can perceive.
- **Observation Model (Z)**: A function $Z(s', a, o) = P(o \mid s', a)$ that defines the probability of observing ?*o* after taking action *a* and ending up in state *s'*.
- **Rewards (R)**: A function $R(s, a)$ that assigns a numerical reward to taking action *a* in state *s*.
- **Discount Factor (*γ*)**: A factor between 0 and 1 that discounts future rewards, reflecting the preference for immediate rewards over future gains.

Mathematical Framework of Partially Observable Markov Decision Process

The decision process in a POMDP is a cycle of states, actions, and observations. At each time step, the agent:

1. Observes a signal that partially reveals the state of the environment.
2. Chooses an action based on the accumulated observations.
3. Receives a reward dependent on the action and the underlying state.
4. Moves to a new state based on the transition model.

The key challenge in a POMDP is that the agent does not know its exact state but has a belief or probability distribution over the possible states. This belief is updated using the Bayes' rule as new observations are made, forming a belief update rule:

$$Bel(s') = \frac{P(o \mid s', a) \sum_s \quad P(s' \mid s, a) Bel(s)}{P(o \mid a, Bel)}$$

Where:

- Bel(s) is the prior belief of being in state s.
- Bel(s') is the updated belief after observing o and taking action a.

Strategies for Solving Partially Observable Markov Decision Processes

Partially Observable Markov Decision Processes (POMDPs) pose significant challenges in environments where agents have incomplete information. Solving POMDPs involves optimizing decision-making strategies under uncertainty, crucial in many real-world applications. This overview highlights key strategies and methods for addressing these challenges.

Belief State Representation:

In POMDPs, agents maintain a belief state—a probability distribution over all possible states—to manage uncertainty. This belief updates dynamically with actions and observations via Bayes' rule.

Solving Techniques:

1. **Value Iteration**: Extends traditional value iteration to belief states, using a piecewise linear and convex value function to calculate the expected rewards and update beliefs accordingly.
2. **Point-Based Methods**: These methods, such as Perseus and Point-Based Value Iteration (PBVI), focus on a select set of belief points to simplify computations and efficiently approximate the value function.

3. **Policy Search Methods**: Methods like QMDP and Finite-state controllers (FIB) search for optimal policies, sometimes simplifying the problem by assuming full observability post-action or using a finite set of controller states.
4. **Monte Carlo Methods**: Techniques like Partially Observable Monte Carlo Planning (POMCP) and Despot leverage Monte Carlo simulations within a tree search framework to estimate policy values under uncertainty, focusing on key scenarios to reduce complexity.

Markov Decision Process vs POMDP

Aspect	Fully observable MDP	Partially Observable MDP (POMDP)
Agent's Knowledge of state	Complete knowledge of the current state of the environment.	Incomplete information about the current state of the environment.
Information of state	The agent knows exactly where it is and what the environment looks like at any given time.	The agent receives observations that are noisy or incomplete indications of the true state.
Uncertainty	No uncertainty about state transitions or outcomes of actions.	Observations influenced by the underlying state, but may not fully reveal it.
Example	A game of chess in which both players possess complete information about the positions of all pieces on the board.	Robot navigating in a foggy environment where it can only see objects within a limited range due to reduced visibility. The robot obtains sensor readings that offer limited information about its surroundings, without possessing direct awareness of the complete environment.

In

conclusion, the Partially Observable Markov Decision Process (POMDP) serves as a robust framework for decision-making in environments characterized by uncertainty and incomplete information. Through simulations like the maze navigation example, we can see how POMDPs effectively model real-world challenges by incorporating uncertainty into the decision process. This enhances our ability to develop intelligent systems capable of operating effectively in complex, dynamic settings. As such, POMDPs are invaluable in advancing the fields of robotics, autonomous systems, and other areas requiring sophisticated decision-making capabilities under uncertainty.